

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

COURSE NAME: ARTIFICIAL
INTELLIGENCE
AND
EXPERT SYSTEMS

COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Modelling Comparative Analysis Assignment

Code of Conduct:

I Juhi Surin (192525148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Juhi Surin

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
-------	----------------	------------------

1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				
Suitability for the Problem				

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

AUTOMOBILE FAULT DIAGNOSIS — COMPARATIVE MODELLING ANALYSIS

Production Rules vs Propositional Logic vs First-Order Logic vs Prolog

Course: Artificial Intelligence and Expert Systems (MLA01) | Tool: SWI-Prolog

1. Problem Statement and Objectives

An automobile service center receives customer complaints such as engine overheating, starting failure, abnormal noise, low mileage, and warning-light activation, and must determine the probable vehicle fault so technicians can act quickly and consistently. The objective of this assignment is to model this single diagnostic problem using four distinct knowledge-representation approaches — production rules, propositional logic, first-order logic, and Prolog — and to compare their expressiveness, inference mechanism, complexity, and practical suitability for deployment in a real service center decision-support tool.

- Domain: automotive fault diagnosis at a vehicle service center.
- Decision requirement: map a set of observed symptoms to the most probable fault and a recommended action.
- Objective: build and compare four equivalent knowledge models of the same rule set, then implement and test the Prolog version.

2. Domain Knowledge and Facts

Entities: Vehicle, Symptom, Fault. Attributes/facts per vehicle: engine_overheating, starting_failure, abnormal_noise, low_mileage, warning_light_on, coolant_leak, battery_weak. Conclusions: fault type — cooling_system_fault, battery_fault, engine_fault, fuel_system_fault — plus a recommended service action.

- Cooling system fault: engine_overheating + coolant_leak.
- Battery fault: starting_failure + warning_light_on + battery_weak.
- Engine (mechanical) fault: abnormal_noise + low_mileage.
- Fuel system fault: starting_failure + low_mileage + (no battery_weak).

3. Production Rule Model

The domain is represented as flat IF–THEN production rules. Each rule fires when all antecedent conditions hold for a vehicle, directly producing the consequent fault and action.

Rule	Condition (IF)	Conclusion (THEN)
R1	engine_overheating AND coolant_leak	fault = cooling_system_fault

R2	starting_failure AND warning_light_on AND battery_weak	fault = battery_fault
R3	abnormal_noise AND low_mileage	fault = engine_fault
R4	starting_failure AND low_mileage AND NOT battery_weak	fault = fuel_system_fault

This model is intuitive and directly readable by technicians but has no variables — a separate rule instance is conceptually needed per vehicle, and it cannot express general relationships between vehicles and components.

4. Propositional Logic Model

Let P = engine_overheating, Q = coolant_leak, R = starting_failure, S = warning_light_on, T = battery_weak, U = abnormal_noise, V = low_mileage. Faults are propositions: A = cooling_system_fault, B = battery_fault, C = engine_fault, D = fuel_system_fault.

- $(P \wedge Q) \rightarrow A$
- $(R \wedge S \wedge T) \rightarrow B$
- $(U \wedge V) \rightarrow C$
- $(R \wedge V \wedge \neg T) \rightarrow D$

Given facts $P = \text{true}$ and $Q = \text{true}$, modus ponens directly derives $A = \text{true}$. Propositional logic correctly captures the same rules as truth-functional implications, but every vehicle needs its own separate set of propositions (P_1, Q_1 for vehicle 1; P_2, Q_2 for vehicle 2, ...), since propositional logic has no variables to generalize 'for any vehicle'.

5. First-Order Logic Model

First-order logic reintroduces variables, predicates, and quantifiers, allowing one general statement to apply to every vehicle:

- $\forall v \text{ EngineOverheating}(v) \wedge \text{CoolantLeak}(v) \rightarrow \text{Fault}(v, \text{cooling_system_fault})$
- $\forall v \text{ StartingFailure}(v) \wedge \text{WarningLightOn}(v) \wedge \text{BatteryWeak}(v) \rightarrow \text{Fault}(v, \text{battery_fault})$
- $\forall v \text{ AbnormalNoise}(v) \wedge \text{LowMileage}(v) \rightarrow \text{Fault}(v, \text{engine_fault})$
- $\forall v \text{ StartingFailure}(v) \wedge \text{LowMileage}(v) \wedge \neg \text{BatteryWeak}(v) \rightarrow \text{Fault}(v, \text{fuel_system_fault})$

With the fact $\text{EngineOverheating}(\text{car_a}) \wedge \text{CoolantLeak}(\text{car_a})$, universal instantiation binds $v = \text{car_a}$ and modus ponens derives $\text{Fault}(\text{car_a}, \text{cooling_system_fault})$. FOL is strictly more expressive than propositional logic here because a single quantified rule now applies to every vehicle without duplication.

6. Prolog Model and Implementation

The FOL statements above translate almost directly into Prolog Horn clauses, since Prolog is a practical, executable subset of first-order logic restricted to definite clauses.

% ----- FACT BASE -----


```

symptom(car_a, engine_overheating).
symptom(car_a, coolant_leak).

symptom(car_b, starting_failure).
symptom(car_b, warning_light_on).
symptom(car_b, battery_weak).

symptom(car_c, abnormal_noise).
symptom(car_c, low_mileage).

symptom(car_d, starting_failure).
symptom(car_d, low_mileage).

% ----- PRODUCTION RULES -----
fault(V, cooling_system_fault) :-
    symptom(V, engine_overheating), symptom(V, coolant_leak).
fault(V, battery_fault) :-
    symptom(V, starting_failure), symptom(V, warning_light_on),
    symptom(V, battery_weak).
fault(V, engine_fault) :-
    symptom(V, abnormal_noise), symptom(V, low_mileage).
fault(V, fuel_system_fault) :-
    symptom(V, starting_failure), symptom(V, low_mileage),
    \+ symptom(V, battery_weak).

?- fault(car_a, X).
X = cooling_system_fault.

?- fault(car_d, X).
X = fuel_system_fault.

```

7. Forward and Backward Chaining Demonstration

7.1 Forward Chaining

Starting from the facts of `car_a` (`engine_overheating`, `coolant_leak`), the engine scans all rule bodies data-first: R1's body is fully satisfied, so `cooling_system_fault` is asserted as a new derived fact — an event-driven, bottom-up derivation.

```

forward(V) :- findall(F, fault(V,F), L),
    format('Forward chaining for ~w: ~w~n', [V, L]).
?- forward(car_a).
Forward chaining for car_a: [cooling_system_fault]

```

7.2 Backward Chaining

Given the hypothesis 'is `car_d` a `fuel_system_fault`?', Prolog works goal-first: it tries to prove `fault(car_d, fuel_system_fault)` by satisfying `starting_failure`, `low_mileage`, and the negation of `battery_weak` in turn against the fact base, backtracking to the next rule only if a conjunct fails.

```

?- fault(car_d, fuel_system_fault).

```

```
true. % goal proven by verifying each sub-condition backward
```

8. Comparative Analysis of Modelling Approaches

The table below compares all four models on the parameters required by the assignment (compulsory comparative table).

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation	Flat IF-THEN rule list	Symbols + logical connectives	Predicates, variables, quantifiers	Facts + Horn-clause rules
Expressiveness	Moderate — no variables/quantifiers	Low — cannot express relations/objects	High — expresses objects, relations, quantifiers	High — FOL subset executable directly
Rule Representation	Direct IF-THEN pairs	Propositional formulas (implications)	Quantified predicate formulas	Horn clauses (Head :- Body)
Inference Mechanism	Manual/algorithmic pattern matching	Truth-table / resolution on propositions	Unification + resolution over predicates	Built-in SLD resolution (unification+backtracking)
Forward Chaining	Simple to apply, scans all rules	Possible but verbose for many propositions	Supported via generalized modus ponens	Directly via findall/recursive rule evaluation
Backward Chaining	Possible manually, no built-in engine	Possible but cumbersome without variables	Supported theoretically via resolution	Native — Prolog's default execution strategy
Handling Relationships	Weak — no relations between entities	None — atomic propositions only	Strong — relations via n-ary predicates	Strong — inherited directly from FOL
Scalability	Poor for large rule sets (no variables)	Poor — combinatorial explosion of propositions	Good — variables generalize over entities	Good — compact code scales via variables
Explainability	High — rules read like human logic	Moderate — logical steps traceable	High — quantified statements are clear	High — trace via proof tree / explain predicates
Ease of Implementation	Easy to write, hard to scale	Easy for small problems only	Requires formal logic training	Easy once FOL is understood; concise code
Suitability for the Problem	Good for simple diagnostic rule sets	Not suitable — cannot generalize over vehicles	Good for formal specification	Best — directly executable, most suitable

9. Results, Discussion and Recommended Model

Test Symptom Set	Expected Fault	Actual Output	Result
car_a: engine_overheating, coolant_leak	cooling_system_fault	cooling_system_fault	Pass

car_b: starting_failure, warning_light_on, battery_weak	battery_fault	battery_fault	Pass
car_c: abnormal_noise, low_mileage	engine_fault	engine_fault	Pass
car_d: starting_failure, low_mileage	fuel_system_fault	fuel_system_fault	Pass
car_e: low_mileage only	No fault (insufficient evidence)	No fault found	Pass

All test vehicles were diagnosed correctly (100% pass rate). Among the four approaches, Prolog is recommended as the most suitable model for this real-world problem: it retains the full expressiveness and relational generality of first-order logic, while additionally providing a working inference engine (unification, resolution, backtracking) out of the box, so no separate reasoning algorithm needs to be built as would be required for the propositional or FOL models on paper. Production rules alone lack variables and do not scale across many vehicles; propositional logic cannot generalize over the vehicle domain at all; FOL is expressive but not directly executable. Prolog therefore combines FOL's expressiveness with production-rule-style readability and a ready inference engine, at the cost of a learning curve for staff unfamiliar with logic programming.

10. Conclusion, References and GitHub Repository

This assignment modelled a single automobile fault-diagnosis problem across four representation paradigms, showing how expressiveness and inference capability increase from production rules through propositional logic to first-order logic, and how Prolog operationalizes first-order logic into a directly executable, testable expert system. All facts, rules, and test queries reproduced in this report are hosted at the project's GitHub repository ([github.com/<username>/auto-fault-diagnosis-comparative](https://github.com/username/auto-fault-diagnosis-comparative)), which contains the Prolog source file, test cases, and this report.

References

- Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- Bratko, I. (2011). Prolog Programming for Artificial Intelligence. Pearson Education.
- Giarratano, J. C., & Riley, G. (2005). Expert Systems: Principles and Programming. Course Technology.
- SWI-Prolog Reference Manual. https://www.swi-prolog.org/pldoc/doc_for?object=manual